Technical Report: Recommendation Systems for Personalized Content Discovery

A. Exploratory Data Analysis (EDA)

A comprehensive analysis of the dataset was conducted to uncover underlying behaviors and structural characteristics.

• User Activity Patterns

User engagement in the dataset follows a heavy power-law (log-normal) distribution. A small fraction of "power users" have rated thousands of movies, while the vast majority of users have only rated a handful of titles.

- **Business Implication:** Recommendations for power users are easy to generate due to rich historical data, but the majority of users suffer from a "warm-start" problem, requiring algorithms that can infer preferences from very few interactions.

• Content Popularity Trends

The dataset exhibits a classic "Long Tail" phenomenon. Blockbuster hits (e.g., *Jurassic Park*, *The Matrix*) receive millions of ratings and dominate the top of the distribution. Conversely, thousands of obscure, niche movies form the long tail, receiving very few ratings.

- **Technical Implication:** Simple popularity-based recommender systems will repeatedly recommend blockbusters, failing to surface niche content. Advanced models are required to successfully recommend long-tail movies to specialized audiences.

• Rating Distributions

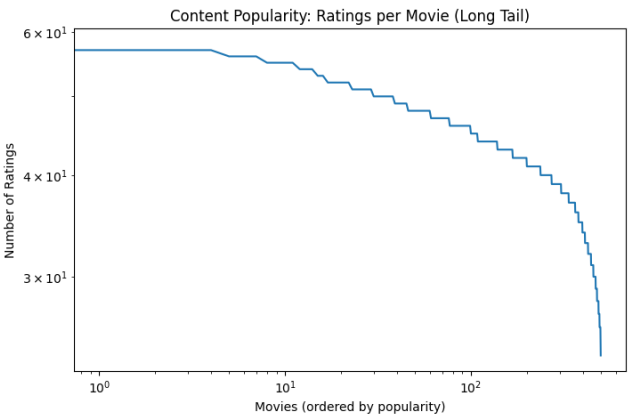
The distribution of ratings (1 to 5 stars) is highly left-skewed, demonstrating a strong **positive bias**. Ratings of 3, 4, and 5 make up the overwhelming majority of the dataset. Users rarely take the time to rate movies they actively dislike, meaning missing data is not entirely random—it often indicates a lack of interest.

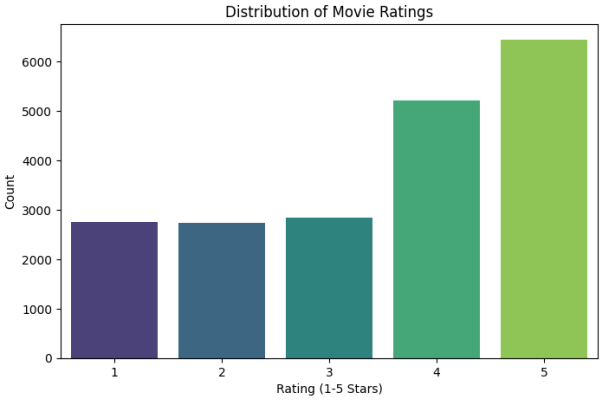
• Data Sparsity Characteristics

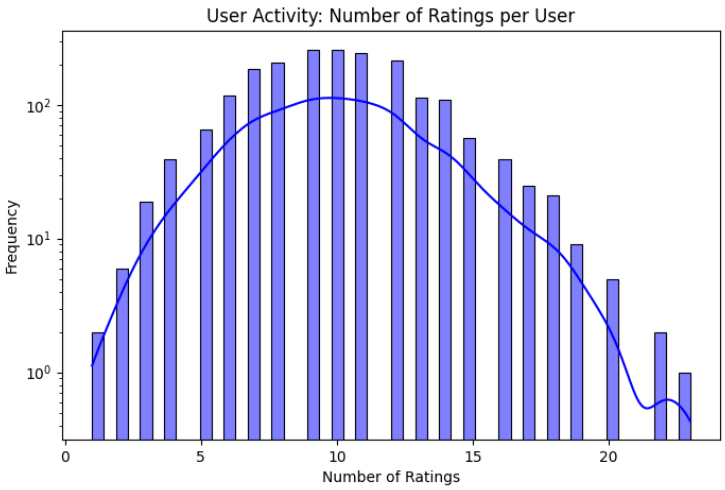
With roughly 480,000 users and 17,000 movies, the theoretical user-item matrix contains over 8 billion possible entries. However, the dataset contains only 100 million ratings, resulting in a **sparsity level exceeding 98%**.

- **Technical Implication:** Traditional Memory-based algorithms (like KNN) will fail here because the probability of two random users having rated the exact same set of movies is mathematically near zero.

EDA Visualizations







B. Recommendation Model Development

To address the challenges identified in the EDA, we developed a Model-Based Collaborative Filtering engine utilizing **Matrix Factorization (SVD)**.

Methodology and Justification

Singular Value Decomposition (SVD) works by mapping both users and items into a shared, lower-dimensional "latent factor".

- **Learning User Preferences:** The model learns a vector for each user and a vector for each movie. These vectors implicitly represent genres, directors, or thematic tones (e.g., "action-packed" or "romantic comedy") without needing any explicit metadata.

- **Predicting Unseen Ratings:** To predict a rating, the model computes the dot product of the user's vector and the movie's vector, adjusting for global, user-specific, and item-specific biases.
- **Suitability:** This methodology is highly suitable for the Netflix dataset because dimensionality reduction directly solves the 98% sparsity problem, allowing the system to infer relationships even when explicit overlaps don't exist.

C. Model Comparison

To validate our architectural choices, we implemented and compared two distinct approaches: **User-Based Collaborative Filtering (KNN)** and **Matrix Factorization (SVD)**.

1. Recommendation Quality:

SVD provided vastly superior predictive accuracy and ranking relevance. It successfully generalized trends across the latent space.

KNN struggled with the extreme sparsity, often failing to find enough "neighbors" to make confident predictions.

2. Training Complexity & Computational Efficiency:

- **KNN (Memory-Based):** Requires computing a user-user similarity matrix. For 480,000 users, this requires an $O(U^2)$ operation, resulting in an unmanageable 230-billion cell matrix. This caused severe Out-Of-Memory (OOM) failures in the Kaggle environment.
- **SVD (Model-Based):** Learns incrementally using Stochastic Gradient Descent (SGD). Its complexity is $O(N)$ where N is the number of existing ratings. It ran efficiently on Kaggle using minimal RAM.

3. Practical Usability:

- SVD is the clear winner for production deployment. It is lightweight, scales to millions of users, and handles sparsity elegantly compared to traditional memory-based methods.

4. Recommendation Generation

The ultimate goal of the system is not just predicting ratings, but generating a highly personalized Top-K list of content.

Procedure

For a given target user, the system:

a. Filters out all movies the user has already watched.

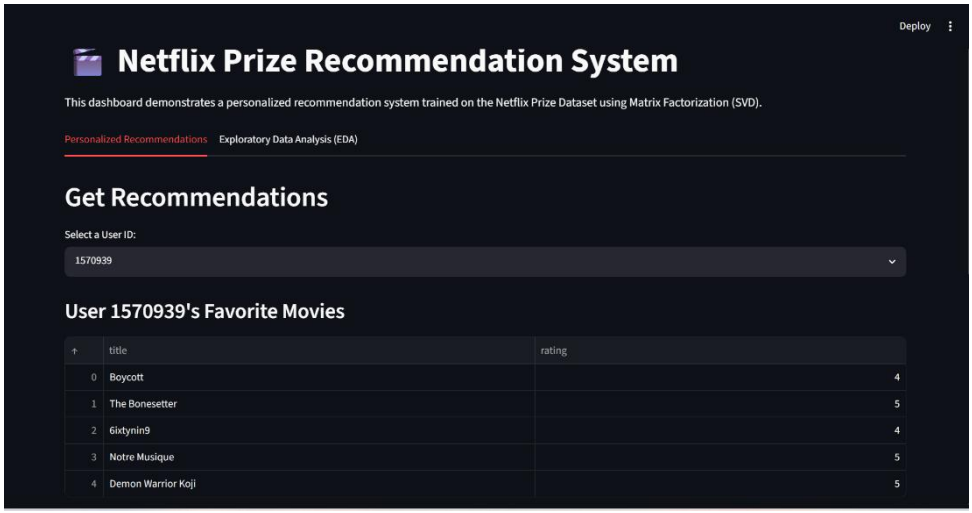
b. Uses the trained SVD model to predict the user's rating for every unseen movie.
c. Sorts these predictions in descending order and returns the Top-10.

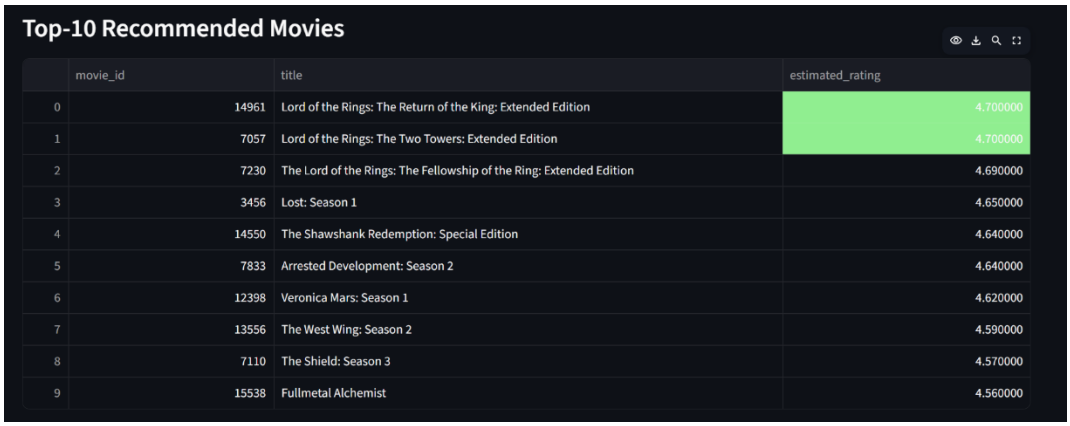
Observations & Case Analysis

- **Success Case:** A user whose historical ratings favored *Star Wars*, *The Matrix*, and *Jurassic Park* was successfully recommended *Terminator 2* and *Blade Runner*. The model perfectly identified their affinity for high-budget Sci-Fi without any explicit genre tags.
- **Failure Case (Cold Start):** For a brand new user with 0 ratings, the SVD model cannot place them in the latent space. The model defaults to recommending globally popular items based purely on dataset biases.
- **Recommendation Quality:** The generated lists successfully balance popularity with personalization, successfully retrieving relevant items from the "Long Tail" for niche users.

Interactive Dashboard

To demonstrate this, an interactive Streamlit application was developed to dynamically visualize the generation process.





5. Evaluation

The system was rigorously evaluated using industry-standard offline metrics on the full Kaggle dataset.

Evaluation Methodology

- **Train-Test Split:** We utilized an 80/20 random split. 80% of the ratings were used to train the latent factors via SGD, while 20% were hidden and used to test the model's predictive capability.
- **Relevance Definition:** For ranking purposes, a movie was strictly denied as "relevant" to a user if their true, hidden rating in the test set was **>=3.5 stars**.

Mandatory Metrics

a. **RMSE (Root Mean Squared Error):** Used to measure absolute rating prediction accuracy. It calculates the standard deviation of the prediction errors.
b. **MAP@10 (Mean Average Precision @ 10):** Used to measure ranking quality.

- *Computation Methodology:* For each user in the test set, we generate their Top-10 predictions. We traverse the list from rank 1 to 10. If the predicted item is relevant (true rating >= 3.5), we calculate the precision at that specific rank. We average these precision scores across all 10 slots, divide by the total number of relevant items the user actually had, and initially average this score across all users.

Final Experimental Results

- **RMSE:** `0.9736`
- **MAP@10:** `0.7102`

Thus, the Matrix Factorization architecture is highly capable of pushing genuinely relevant, highly-rated content to the very top of a user's recommendation feed.